

MotorGest - Rapport de conception

Projet POO avancée - Bachelor 3 Info - Kalotta, Mathéo, Yohan, Louis

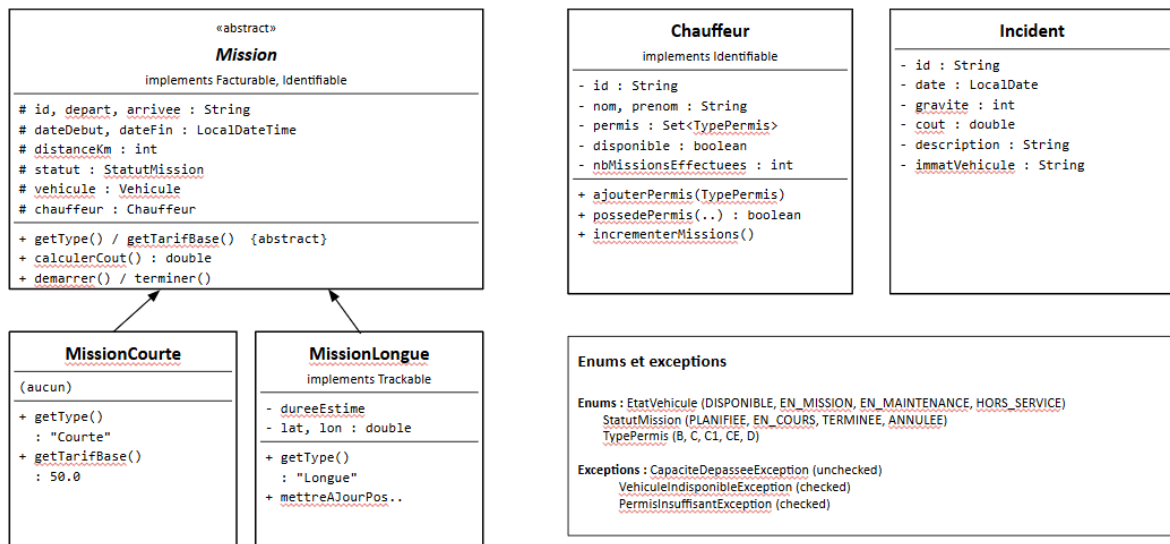
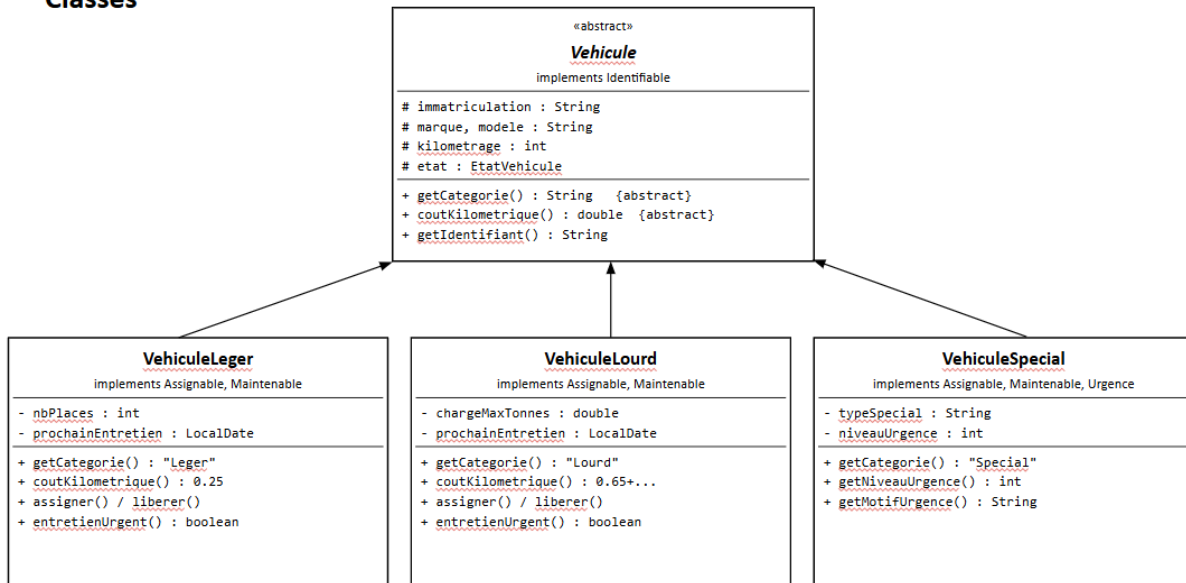
1. Diagramme de classes

Le schéma ci-dessous présente les classes principales et leurs relations. Les classes en italique sont abstraites. Les interfaces sont préfixées de <<I>>.

Interfaces



Classes



Registre<T extends Identifiable> (package util)
<pre> - elements : Map<String, T> + ajouter(T) + supprimer(String) : boolean + trouver(String) : T + tout() : List<T> + filtrer(Predicate<? super T>) + contient(String) : boolean + taille() : int + values() : Collection<T> </pre>

GestionnaireFlotte (package controller)
<pre> - vehicules : Registre<Vehicule> - chauffeurs : Registre<Chauffeur> - missions : Registre<Mission> - incidents : List<Incident> - filePriorite : PriorityQueue<Mission> + ajouter / supprimer / trouver (Vehicule, Chauffeur, Mission) + affecterMission(idM, immat, idCh) throws VehiculeIndisponibleException, PermisInsuffisantException + terminerMission(String) + rechercherVehicules(..., ..., ...) + rechercherMissions(..., ..., ...) + vehiculesDisponibles() : List<Vehicule> + chauffeursDisponibles() : List<Chauffeur> + vehiculesAEntretenir() : List<Vehicule> </pre>

2. Justification des choix OO

Vehicule et Mission sont des classes abstraites parce qu'il existe une notion partagée (immatriculation, marque, modèle pour les véhicules ; id, départ, arrivée pour les missions) mais on ne veut jamais instancier directement la classe parente. Les méthodes abstraites getCategory() et coutKilometrique() pour Vehicule, getType() et getTarifBase() pour Mission, forcent chaque sous-classe à fournir son comportement spécifique.

Les capacités transversales sont des interfaces (Assignable, Maintenable, Trackable, Urgence, Facturable) parce qu'elles peuvent être combinées librement. VehiculeSpecial implémente trois interfaces à la fois (Assignable + Maintenable + Urgence), ce qui serait impossible avec de l'héritage multiple en Java. Si on avait modélisé ces capacités comme des classes parentes, on aurait eu une explosion combinatoire de sous-classes.

L'interface Identifiable permet de définir un generic borné Registre<T extends Identifiable>. Toutes les entités principales l'implémentent, ce qui évite de dupliquer le code de gestion des collections (CRUD, recherche par ID). Le Registre utilise getIdentifiant() comme clé et accepte un Predicate<? super T> pour le filtrage (wildcard super qui suit le principe PECS).

Trois exceptions custom : VehiculeIndisponibleException (checked) et PermisInsuffisantException (checked) correspondent à des situations métier prévisibles que l'appelant DOIT gérer. CapacitéDepasseeException est unchecked car elle signale une violation d'invariant programmatique (id déjà existant), pas un cas métier.

3. Difficultés rencontrées et solutions

La principale difficulté a été de bien découpler le modèle de la vue. Au départ on mettait de la logique métier dans les ActionListener (vérifier la disponibilité d'un véhicule directement dans le bouton d'affectation). On a refactorisé pour déplacer toute cette logique dans `GestionnaireFlotte.affecterMission()`, qui lève les exceptions appropriées, et la vue se contente de les attraper pour afficher un message.

Le filtrage avec plusieurs critères optionnels (certains nuls) posait un problème de lisibilité. On a adopté un pattern de `filter()` chaînée où chaque filter teste d'abord si le critère est null ou vide. C'est plus lisible que des if imbriqués.

La synchronisation entre les onglets quand une donnée change : terminer une mission doit libérer le véhicule, incrémenter le compteur du chauffeur, et donc rafraîchir trois onglets différents. On a résolu ça avec une interface `Rafraichissable` que tous les panneaux implémentent, et une méthode `rafraichirTout()` sur la fenêtre principale qui parcourt les onglets via `SwingUtilities.invokeLater()`.

4. Améliorations possibles

Avec plus de temps, on aurait aimé ajouter :

- Une vraie persistance via sérialisation Java pour sauvegarder aussi l'état des missions (affectations, statuts) qui n'est pas conservé dans les CSV
- Une carte des positions pour les `MissionLongue` qui implémentent déjà `Trackable` mais dont on n'exploite pas encore les coordonnées dans l'UI
- Une vue calendrier pour visualiser le planning hebdomadaire des chauffeurs
- Des tests unitaires JUnit pour les classes du controller